



Les commandes SQL classiques

1. SELECT

Permet de récupérer des données.

```
SELECT *
```

```
FROM Person.Person;
```

Sélection de colonnes :

```
SELECT
```

```
    FirstName,
```

```
    LastName
```

```
FROM Person.Person;
```

👉 * = toutes les colonnes.

2. FROM

Indique la table dans laquelle on cherche.

```
SELECT *
```

```
FROM Production.Product;
```

Sans FROM, SQL Server ne sait pas où chercher (sauf pour des expressions comme SELECT GETDATE();).

3. WHERE

Filtre les lignes.

```
SELECT *
```

```
FROM Production.Product
```

```
WHERE ListPrice > 100;
```

Plusieurs conditions :

```
WHERE Color = 'Red'
```

```
AND ListPrice < 500;
```

4. ORDER BY

Trie les résultats.

Croissant :

```
ORDER BY ListPrice ASC;
```

Décroissant :

```
ORDER BY ListPrice DESC;
```

Plusieurs colonnes :

```
ORDER BY Color, Name;
```

5. INSERT

Ajoute des données.

```
INSERT INTO Employe  
(  
    Nom,  
    Age  
)  
VALUES  
(  
    'Alice',  
    30  
);
```

Plusieurs lignes :

```
INSERT INTO Employe  
VALUES  
( 'Alice', 30 ),  
( 'Bob', 25 );
```

6. UPDATE

Modifie des données.

```
UPDATE Employe  
SET Age = 31  
WHERE Nom = 'Alice';
```

⚠ Sans WHERE :

```
UPDATE Employe  
SET Age = 31;
```

➡ Toutes les lignes sont modifiées.

7. DELETE

Supprime des lignes.

```
DELETE  
FROM Employe  
WHERE Nom = 'Alice';
```

⚠ Sans WHERE :

Toutes les lignes sont supprimées.

8. CREATE TABLE

Créer une table.

```
CREATE TABLE Employe  
(  
    Id INT,  
    Nom VARCHAR(50)  
);
```

9. ALTER TABLE

Modifier une table.

Ajouter une colonne :

```
ALTER TABLE Employe  
ADD Telephone VARCHAR(20);
```

Supprimer une colonne :

```
ALTER TABLE Employe  
DROP COLUMN Telephone;
```

10. DROP

Supprime complètement un objet.

```
DROP TABLE Employe;
```

ou

```
DROP PROCEDURE MaProcedure;
```

ou

```
DROP FUNCTION MaFonction;
```

11. JOIN

Permet de relier plusieurs tables.

```
SELECT  
    p.FirstName,  
    e.JobTitle  
FROM Person.Person p  
JOIN HumanResources.Employee e  
ON p.BusinessEntityID = e.BusinessEntityID;
```

Types les plus courants :

INNER JOIN → uniquement les correspondances.

LEFT JOIN → toutes les lignes de gauche, même sans correspondance.

RIGHT JOIN → toutes les lignes de droite.

FULL JOIN → toutes les lignes des deux tables.

12. GROUP BY

Regroupe des lignes.

```
SELECT
    Color,
    COUNT(*) AS NbProduits
FROM Production.Product
GROUP BY Color;
```

Résultat :

Color	NbProduits
Red	10
Black	25

13. HAVING

Filtre les groupes.

```
SELECT
    Color,
    COUNT(*) AS Nb
FROM Production.Product
GROUP BY Color
HAVING COUNT(*) > 10;
```

💡 Différence :

- WHERE → filtre les lignes avant le regroupement.
- HAVING → filtre les groupes après le regroupement.

14. Les fonctions d'agrégation

Compter :

COUNT(*)

Moyenne :

AVG(ListPrice)

Somme :

SUM(ListPrice)

Minimum :

MIN(ListPrice)

Maximum :

MAX(ListPrice)

15. DISTINCT

Supprime les doublons.

```
SELECT DISTINCT Color
FROM Production.Product;
```

★ Les commandes SQL à connaître par cœur

Commande	Sert à...
SELECT	Lire des données
FROM	Choisir la table
WHERE	Filtrer
ORDER BY	Trier
INSERT	Ajouter
UPDATE	Modifier
DELETE	Supprimer
CREATE TABLE	Créer une table
ALTER TABLE	Modifier une table
DROP	Supprimer un objet
JOIN	Relier plusieurs tables
GROUP BY	Regrouper
HAVING	Filtrer des groupes
COUNT, SUM, AVG, MIN, MAX	Calculer des statistiques
DISTINCT	Supprimer les doublons



Partie 1 : Les bases

1. Qu'est-ce que le T-SQL ?

Le T-SQL (Transact-SQL) est le langage utilisé par Microsoft SQL Server.

Il permet non seulement de faire des requêtes (SELECT, INSERT, UPDATE...) mais aussi de programmer directement dans la base de données :

- procédures stockées
- fonctions
- triggers
- boucles
- variables
- conditions

👉 En résumé :

SQL = interroger la base

T-SQL = programmer dans la base

Le but est d'automatiser des traitements directement côté serveur.

2. BEGIN ... END

Permet de regrouper plusieurs instructions.

Sans BEGIN...END, seul le premier ordre appartient au IF, WHILE, etc.

Exemple

```
IF @Age >= 18
```

```
BEGIN
```

```
    PRINT 'Majeur'
```

```
    PRINT 'Accès autorisé'
```

```
END
```

À retenir :

➡ Toujours utiliser BEGIN...END, même pour une seule instruction.

3. GO

Le GO n'est pas une commande SQL.

C'est une instruction comprise par SQL Server Management Studio qui signifie :

"Exécute maintenant tout ce qui précède."

Sans GO, plusieurs instructions peuvent être envoyées au serveur comme un seul lot.

Exemple

```
CREATE TABLE Test
(
    Id INT
);
GO
```

```
INSERT INTO Test
VALUES (1);
GO
```

À retenir :

✅ Toujours mettre un GO après :

- CREATE PROCEDURE
- CREATE FUNCTION
- CREATE TRIGGER
- CREATE TYPE

4. EXEC

EXEC sert à exécuter :

- une procédure
- du SQL dynamique

Exemple

```
EXEC dbo.usp_AfficherDateHeure;
```

Avec paramètres :

```
EXEC dbo.usp_RechercheProduits
    @Categorie='Bikes',
    @PrixMax=500;
```

5. SET

SET permet principalement :

- d'affecter une valeur à une variable
- de modifier certains paramètres SQL Server

Exemple

```
SET @Age = 25;
```

Autres exemples :

```
SET NOCOUNT ON;
```

```
SET DATEFORMAT dmy;
```

6. PRINT

Affiche un message dans l'onglet Messages.

Exemple

```
PRINT 'Bonjour';
```

Avec une variable :

```
PRINT @Nom;
```

Concaténation :

```
PRINT 'Nom : ' + @Nom;
```

⚠ Si tu concatènes un entier :

```
PRINT 'Age : ' + CAST(@Age AS VARCHAR(10));
```

7. Tables temporaires

Une table temporaire commence toujours par :

```
#MaTable
```

Elle est supprimée automatiquement à la fin de la session.

Exemple

```
CREATE TABLE #Temp
```

```
(
```

```
    Nom VARCHAR(50)
```

```
);
```

```
INSERT INTO #Temp
```

```
VALUES ('Nyx');
```

```
SELECT *
```

```
FROM #Temp;
```

```
DROP TABLE #Temp;
```

8. OBJECT_ID()

Permet de savoir si un objet existe.

Très utilisé avant un DROP.

Exemple

```
IF OBJECT_ID('dbo.MaProcedure','P') IS NOT NULL
    DROP PROCEDURE dbo.MaProcedure;
```

Les lettres importantes :

Lettre	Objet
P	Procédure
FN	Fonction
U	Table

9. TYPE_ID()

Vérifie si un type existe.

Exemple :

```
IF TYPE_ID('EmployeType') IS NOT NULL
    DROP TYPE EmployeType;
```

10. COL_LENGTH()

Vérifie si une colonne existe déjà.

Exemple :

```
IF COL_LENGTH('dbo.Employes','Province') IS NULL
BEGIN
    ALTER TABLE dbo.Employes
    ADD Province VARCHAR(50);
END
```

11. GETDATE()

Retourne la date et l'heure actuelles.

Exemple :

```
SELECT GETDATE();
```

Très utilisé dans les TP :

```
SET ModifiedDate = GETDATE();
```

12. @@ROWCOUNT

Retourne le nombre de lignes modifiées par la dernière instruction.

Exemple :

```
UPDATE Person.Person  
SET ModifiedDate = GETDATE();  
  
PRINT @@ROWCOUNT;
```

13. SET NOCOUNT ON

Évite d'afficher :
(15 lignes affectées)
On l'utilise quasiment dans toutes les procédures.
Exemple :
CREATE PROCEDURE MaProcedure
AS
BEGIN

SET NOCOUNT ON;

...
END

★ À connaître par cœur

Commande	À quoi sert-elle ?
BEGIN...END	Délimite un bloc d'instructions
GO	Exécute le lot courant
EXEC	Exécute une procédure
SET	Affecte une valeur ou modifie une option
PRINT	Affiche un message
GETDATE()	Date et heure actuelles
OBJECT_ID()	Vérifie l'existence d'un objet
TYPE_ID()	Vérifie l'existence d'un type
COL_LENGTH()	Vérifie l'existence d'une colonne
@@ROWCOUNT	Nombre de lignes affectées
SET NOCOUNT ON	Masque "(x lignes affectées)"

💡 Le conseil "prof"

Si tu ne devais retenir que **5 commandes** pour les TP et l'examen, ce seraient celles-ci :

OBJECT_ID()
TYPE_ID()
GETDATE()
@@ROWCOUNT
SET NOCOUNT ON



Partie 2 : Les variables

1. Qu'est-ce qu'une variable ?

Une variable est une zone mémoire qui permet de stocker une valeur afin de la réutiliser plus tard.

Une variable possède toujours :

- un nom
- un type
- une valeur (même si cette valeur est NULL)

Par défaut, une variable vaut NULL.

2. Déclarer une variable

Toutes les variables commencent par @.

La déclaration se fait avec DECLARE.

Exemple

```
DECLARE @Nom VARCHAR(50);
```

```
DECLARE @Age INT;
```

```
DECLARE @DateNaissance DATE;
```

On peut également déclarer plusieurs variables en une seule fois.

```
DECLARE
```

```
    @Nom VARCHAR(50),
```

```
    @Age INT,
```

```
    @Salaire MONEY;
```



À retenir

- Toujours déclarer une variable avant de l'utiliser.
- Une variable n'existe que pendant l'exécution du script.

3. Les principaux types

Les types que tu rencontreras le plus souvent :

Type	Utilisation
INT	Nombre entier
DECIMAL(x,y)	Nombre décimal
MONEY	Argent
VARCHAR(x)	Texte
NVARCHAR(x)	Texte Unicode
DATE	Date
DATETIME	Date + heure
BIT	Vrai/Faux (0 ou 1)

Le cours liste également d'autres types (SMALLINT, FLOAT, UNIQUEIDENTIFIER, etc.), mais ceux-ci sont les plus utilisés dans les TP.

4. Donner une valeur

On utilise généralement SET.

Exemple

```
SET @Nom = 'Nyx';
```

```
SET @Age = 30;
```

```
SET @DateNaissance = '1995-02-15';
```

Les chaînes de caractères sont toujours entre apostrophes.

```
'Bonjour'
```

et non

```
Bonjour
```

5. Affecter une valeur avec SELECT

Très utilisé lorsque la valeur provient d'une table.

```
DECLARE @Nb INT;
```

```
SELECT @Nb = COUNT(*)
```

```
FROM HumanResources.Employee;
```

Ou encore :

```
DECLARE @Nom VARCHAR(50);
```

```
SELECT @Nom = LastName
```

```
FROM Person.Person
```

```
WHERE BusinessEntityID = 1;
```

⚠ La requête doit renvoyer une seule ligne lorsqu'on affecte une valeur à une variable scalaire.

6. NULL

Une variable vaut NULL tant qu'on ne lui affecte pas de valeur.

```
DECLARE @Nom VARCHAR(50);
```

```
PRINT @Nom;
```

n'affichera rien car :

```
@Nom = NULL
```



NULL ≠ 0

NULL signifie :

"valeur inconnue ou absente"

Ce n'est pas un nombre.

7. CONVERT()

Permet de changer le type d'une donnée.

Syntaxe :

```
CONVERT(Type, Valeur)
```

Exemple

Nombre vers texte

```
CONVERT(VARCHAR(10), @Age)
```

Date vers texte

```
CONVERT(VARCHAR(10), GETDATE(), 103)
```

Date complète

```
CONVERT(VARCHAR(19), GETDATE(), 121)
```

On l'utilise énormément dans les PRINT. Le cours insiste aussi sur le fait que SQL Server essaie parfois de convertir automatiquement les types (conversion implicite), mais qu'il est préférable d'utiliser CONVERT lorsque c'est nécessaire.

8. Concaténation

En T-SQL, on utilise :

+

Exemple :

```
PRINT 'Bonjour ' + @Nom;
```



Si un entier est utilisé :

```
PRINT 'Age : ' + @Age;
```

✗ Erreur

Il faut écrire :

```
PRINT 'Age : ' + CONVERT(VARCHAR(10), @Age);
```

Pourquoi ?

Parce que SQL Server interprète naturellement + comme une addition lorsqu'il rencontre un nombre.

9. Variables TABLE

Une variable peut aussi contenir plusieurs lignes.

```
DECLARE @Employes TABLE
```

```
(
```

```
    Id INT,
```

```
    Nom VARCHAR(50)
```

```
);
```

Insertion

```
INSERT INTO @Employes
```

```
VALUES
```

```
(1, 'Alice'),
```

```
(2, 'Bob');
```

Lecture

```
SELECT *
```

```
FROM @Employes;
```

Ces variables sont automatiquement supprimées à la fin de leur utilisation et sont très pratiques pour stocker temporairement un ensemble de données. Le cours souligne également qu'elles sont particulièrement utiles dans les fonctions, procédures et triggers.

10. Table temporaire (#)

À ne pas confondre avec une variable TABLE.

```
CREATE TABLE #Temp
```

```
(
```

```
    Nom VARCHAR(50)
```

```
);
```

Elle existe uniquement pendant la session.

À la fin :

```
DROP TABLE #Temp;
```

La différence principale :

Variable TABLE	Table temporaire
----------------	------------------

@Table	#Table
--------	--------

Créée avec DECLARE	Créée avec CREATE TABLE
--------------------	-------------------------

Supprimée	Disparaît à la fin de la session ou
-----------	-------------------------------------

automatiquement	après DROP TABLE
-----------------	------------------

Le cours présente les deux approches et montre leur utilisation.

★ À connaître par cœur

Commande	Utilité
DECLARE	Déclarer une variable
SET	Donner une valeur
SELECT @Var = ...	Récupérer une valeur depuis une table
CONVERT()	Changer le type d'une donnée
+	Concaténer des chaînes
DECLARE @Table TABLE	Variable TABLE
CREATE TABLE #Temp	Table temporaire



Le conseil "prof"

Si tu ne devais retenir que 5 notions de ce chapitre, ce seraient :

```
DECLARE
```

```
SET
```

```
SELECT @Variable =
```

```
CONVERT()
```

```
DECLARE @Table TABLE
```



Partie 3 : Les contrôles conditionnels

1. À quoi servent les contrôles conditionnels ?

Ils permettent au programme de prendre des décisions.

En fonction d'une condition, SQL Server peut :

- exécuter un bloc de code,
- ignorer ce bloc,
- ou exécuter un autre bloc.

Le cours présente principalement deux outils :

- IF
- CASE

Ils servent à réagir en fonction du contenu d'une variable ou du résultat d'une expression.

2. IF

Le IF permet d'exécuter du code si une condition est vraie.

Syntaxe :

IF condition

BEGIN

...

END

Exemple :

```
DECLARE @Age INT = 20;
```

```
IF @Age >= 18
```

```
BEGIN
```

```
    PRINT 'Majeur';
```

```
END
```

3. ELSE

Si la condition est fausse, on peut utiliser ELSE.

```
DECLARE @Age INT = 15;
```

```
IF @Age >= 18
BEGIN
    PRINT 'Majeur';
END
ELSE
BEGIN
    PRINT 'Mineur';
END
```

⚠ ELSE est facultatif.

4. Les comparateurs

Les principaux opérateurs :

Opérateur	Signification
=	égal
<>	différent
>	supérieur
<	inférieur
>=	supérieur ou égal
<=	inférieur ou égal

Exemple :

```
IF @Salaire > 3000
```

Le cours rappelle que pour tester l'égalité, on utilise un simple = en T-SQL.

5. Les opérateurs logiques

On peut combiner plusieurs conditions.

AND

Toutes les conditions doivent être vraies.

```
IF @Age >= 18
AND @Age < 65
```

OR

Une seule condition doit être vraie.

```
IF @Ville = 'Mons'
OR @Ville = 'Charleroi'
```

NOT

Inverse une condition.

```
IF NOT (@Age > 18)
```

Le cours conseille de mettre les conditions complexes entre parenthèses pour une meilleure lisibilité.

6. CASE simple

Le CASE simple compare une valeur à plusieurs possibilités.

```
CASE @Note
    WHEN 10 THEN 'Passable'
    WHEN 15 THEN 'Bien'
    WHEN 20 THEN 'Excellent'
```

```
END
```

Il fonctionne comme un switch dans d'autres langages.

7. CASE recherché

Le CASE recherché compare des conditions.

```
CASE
    WHEN @Age < 18 THEN 'Mineur'
    WHEN @Age < 65 THEN 'Adulte'
    ELSE 'Senior'
```

```
END
```

Il est beaucoup plus utilisé dans les requêtes SQL.

8. CASE dans un SELECT

Très utile pour transformer une valeur.

Exemple :

```
SELECT
    Nom,
    CASE
        WHEN Salaire < 2000
            THEN 'Faible'
        WHEN Salaire < 4000
            THEN 'Moyen'
        ELSE 'Élevé'
    END AS NiveauSalaire
FROM Employe;
```

Chaque ligne reçoit une catégorie.

9. CASE dans une variable

On peut également stocker le résultat dans une variable.

```
DECLARE @Categorie VARCHAR(20);
```

```
SELECT
```

```
@Categorie = CASE
```

```
    WHEN YEAR(BirthDate) > 2000
```

```
        THEN 'Très jeune'
```

```
    WHEN YEAR(BirthDate) > 1990
```

```
        THEN 'Jeune'
```

```
    ELSE 'Expérimenté'
```

```
END
```

```
FROM HumanResources.Employee
```

```
WHERE BusinessEntityID = 1;
```

Le cours montre que cette approche permet de récupérer directement le résultat du CASE dans une variable.

10. IF ou CASE ?

Utilise IF lorsque tu veux exécuter plusieurs instructions.

Exemple :

```
IF @Age >= 18
```

```
BEGIN
```

```
    UPDATE Person.Person
```

```
    SET ModifiedDate = GETDATE();
```

```
    PRINT 'Mise à jour effectuée';
```

```
END
```

Utilise CASE lorsque tu veux retourner une valeur.

Exemple :

```
SELECT
```

```
Nom,
```

```
CASE
```

```
    WHEN Salaire > 3000
```

```
        THEN 'Bon'
```

```
    ELSE 'Normal'
```

```
END
```

```
FROM Employe;
```

★ Les pièges

Toujours utiliser `BEGIN...END`

Même pour une seule instruction.

`ELSE` n'est pas obligatoire

Mais il est fortement conseillé.

`CASE` n'exécute pas plusieurs instructions

Il retourne uniquement une valeur.

`CASE` est souvent utilisé dans `SELECT`

C'est son utilisation principale dans le cours.

★ À connaître par cœur

Commande	Utilité
<code>IF</code>	Tester une condition
<code>ELSE</code>	Cas contraire
<code>CASE</code>	Retourner une valeur selon une condition
<code>AND</code>	Toutes les conditions vraies
<code>OR</code>	Au moins une condition vraie
<code>NOT</code>	Inverse une condition

🎯 Le conseil "prof"

Si je devais résumer ce chapitre en 3 règles :

1. **`IF`** → exécuter du code.
2. **`CASE`** → retourner une valeur.
3. **Toujours entourer les blocs avec `BEGIN...END`**, même si une seule instruction est concernée.



Partie 4 : Boucles & Curseurs

1. La boucle WHILE

Une boucle permet de répéter plusieurs instructions tant qu'une condition est vraie.

Syntaxe :

```
WHILE condition  
BEGIN
```

...

```
END
```

Exemple :

```
DECLARE @Compteur INT = 1;
```

```
WHILE @Compteur <= 5  
BEGIN  
    PRINT @Compteur;  
    SET @Compteur += 1;
```

```
END
```

Résultat

```
1  
2  
3  
4  
5
```

2. Bien penser à modifier la condition

Une erreur très fréquente :

```
WHILE @Compteur <= 5  
BEGIN
```

```
    PRINT @Compteur;
```

```
END
```

❌ La variable ne change jamais.

Résultat :

➡ boucle infinie.

Toujours faire :

```
SET @Compteur += 1;
```

3. BREAK

Permet de sortir immédiatement d'une boucle.

Exemple

```
WHILE 1 = 1
BEGIN
    PRINT 'Bonjour';
    BREAK;
END
```

Après BREAK, SQL Server quitte directement la boucle.

4. CONTINUE

Passe directement au tour suivant.

Exemple

```
WHILE @Compteur <= 10
BEGIN
    SET @Compteur += 1;
    IF @Compteur = 5
        CONTINUE;
    PRINT @Compteur;
END
```

Le chiffre 5 ne sera jamais affiché.

5. Les curseurs

Un curseur permet de parcourir une requête ligne par ligne.

Contrairement à un SELECT classique qui traite toutes les lignes d'un coup, le curseur traite chaque enregistrement individuellement.

Le cours le présente comme une solution lorsqu'un traitement doit être appliqué à chaque ligne séparément.

6. Les étapes d'un curseur

Il y a toujours le même schéma.

1. Déclarer les variables

```
DECLARE @Nom VARCHAR(50);
```

2. Déclarer le curseur

```
DECLARE MonCurseur CURSOR FOR  
SELECT Nom  
FROM Employe;
```

3. Ouvrir le curseur

```
OPEN MonCurseur;  
Le curseur est prêt à être parcouru.
```

4. Premier FETCH

```
FETCH NEXT  
FROM MonCurseur  
INTO @Nom;  
Cette instruction récupère la première ligne.
```

5. Boucle

```
WHILE @@FETCH_STATUS = 0  
BEGIN  
    PRINT @Nom;  
  
    FETCH NEXT  
    FROM MonCurseur  
    INTO @Nom;  
END
```

Le deuxième FETCH est indispensable pour passer à la ligne suivante.

6. Fermer

```
CLOSE MonCurseur;
```

7. Libérer

```
DEALLOCATE MonCurseur;  
Le curseur disparaît complètement.
```

7. @@FETCH_STATUS

Après chaque FETCH, SQL Server met à jour cette variable système.

Valeur	Signification
--------	---------------

0	Ligne trouvée
---	---------------

-1	Plus de ligne
----	---------------

-2	Ligne inaccessible
----	-----------------------

On écrit presque toujours :

```
WHILE @@FETCH_STATUS = 0
```

8. Structure complète

```
DECLARE @Nom VARCHAR(50);
```

```
DECLARE Cur CURSOR FOR
```

```
SELECT Nom  
FROM Employe;
```

```
OPEN Cur;
```

```
FETCH NEXT  
FROM Cur  
INTO @Nom;
```

```
WHILE @@FETCH_STATUS = 0  
BEGIN  
    PRINT @Nom;  
    FETCH NEXT  
    FROM Cur  
    INTO @Nom;  
END
```

```
CLOSE Cur;  
DEALLOCATE Cur;
```

9. Quand utiliser un curseur ?

✓ Lorsqu'on doit traiter les lignes une par une.

Par exemple :

- envoyer un email à chaque employé
- afficher un message différent pour chaque produit
- effectuer un calcul particulier sur chaque ligne

✗ Pas pour faire des mises à jour globales.

Dans ce cas :

UPDATE ...

est beaucoup plus rapide.

★ Les pièges

Toujours :

- OPEN
- premier FETCH
- WHILE
- deuxième FETCH
- CLOSE
- DEALLOCATE

Oublier une seule de ces étapes provoque généralement une erreur ou un comportement inattendu.

★ À connaître par cœur

Commande	Utilité
WHILE	Répéter des instructions
BREAK	Sortir d'une boucle
CONTINUE	Passer à l'itération suivante
DECLARE CURSOR	Créer un curseur
OPEN	Ouvrir le curseur
FETCH NEXT	Lire la ligne suivante
@@FETCH_STATUS	Vérifier si une ligne existe
CLOSE	Fermer le curseur
DEALLOCATE	Supprimer le curseur

Le conseil "prof"

Le curseur est souvent considéré comme "lourd". En SQL, on privilégie autant que possible les traitements par ensemble (SELECT, UPDATE, DELETE) plutôt que ligne par ligne.

En revanche, si un exercice demande explicitement de parcourir les résultats un à un, le curseur est la bonne solution.

Partie 9 : Les Triggers

1. Qu'est-ce qu'un Trigger ?

Un trigger (déclencheur) est un programme stocké qui s'exécute automatiquement lorsqu'un événement précis se produit dans la base de données.

Contrairement à une procédure, on ne l'appelle jamais avec EXEC. C'est SQL Server qui le lance automatiquement lorsqu'une action correspondante est effectuée.

Le cours précise qu'un trigger est généralement déclenché par un ordre DML (INSERT, UPDATE, DELETE), mais qu'il existe également des triggers DDL et LOGON.

2. Les trois types de Triggers

Trigger DML

Se déclenche lors d'un :

- INSERT
- UPDATE
- DELETE

C'est le type de trigger le plus courant.

Trigger DDL

Se déclenche lorsqu'on modifie la structure de la base :

- CREATE
- ALTER
- DROP
- GRANT
- DENY
- REVOKE
- UPDATE STATISTICS

Trigger LOGON

Se déclenche lors de la connexion d'un utilisateur au serveur SQL.
Le cours précise également que SQL Server utilise principalement des triggers AFTER/FOR ou INSTEAD OF. Les triggers BEFORE n'existent pas tels quels en T-SQL.

3. Syntaxe d'un Trigger DML

```
CREATE TRIGGER NomTrigger
ON MaTable
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    -- Traitement
END;
```

À retenir :

- ON → table concernée
- AFTER → événement déclencheur
- AS → début du code

4. Les tables INSERTED et DELETED

Lorsqu'un trigger s'exécute, SQL Server crée automatiquement des tables temporaires spéciales.

INSERTED

Contient les nouvelles données.

Utilisée après :

- INSERT
- UPDATE

DELETED

Contient les anciennes données.

Utilisée après :

- DELETE
- UPDATE

Cas d'un UPDATE

Lors d'une mise à jour, les deux tables existent simultanément.

Table	Contenu
DELETED	Anciennes valeurs
INSERTED	Nouvelles valeurs

Cela permet de comparer avant/après une modification.

5. Fonction UPDATE()

Le cours présente la fonction :

UPDATE(NomColonne)

Elle permet de vérifier si une colonne particulière a été modifiée lors d'un INSERT ou d'un UPDATE.

Exemple :

```
IF UPDATE(Salaire)
```

```
BEGIN
```

```
    PRINT 'Le salaire a changé';
```

```
END
```

Très pratique lorsqu'un trigger ne doit réagir qu'à certaines colonnes.

6. COMMIT et ROLLBACK dans un Trigger

Le rôle principal d'un trigger est souvent de valider ou refuser une opération.

Le cours insiste sur deux points :

COMMIT

Le COMMIT est effectué automatiquement par SQL Server.

Il n'est généralement pas nécessaire de l'écrire dans un trigger.

ROLLBACK

En revanche, le ROLLBACK est très utilisé.

Il permet d'annuler complètement l'opération qui a déclenché le trigger.

Exemple :

```
IF condition
```

```
BEGIN
```

```
    ROLLBACK;
```

```
END
```

Toute l'opération est annulée.

7. RAISERROR

Permet d'envoyer un message d'erreur personnalisé.

Syntaxe :

```
RAISERROR('Message', Severite, Etat);
```

Exemple :

```
RAISERROR('Opération impossible',16,1);
```

Le message apparaît directement dans SQL Server Management Studio.

8. @@ERROR

Variable système contenant le numéro de la dernière erreur.

Exemple :

```
IF @@ERROR <> 0
```

```
BEGIN
```

```
    RAISERROR('Erreur détectée',16,1);
```

```
    ROLLBACK;
```

```
END
```

Le cours montre qu'on peut utiliser @@ERROR pour décider s'il faut annuler une transaction.

9. Les paramètres de RAISERROR

Le cours détaille les deux paramètres les plus importants.

Sévérité

Détermine l'importance de l'erreur.

Exemple :

```
RAISERROR('Erreur',16,1);
```

Le niveau 16 est celui que l'on rencontre le plus souvent dans les exemples.

État (State)

Correspond à un numéro compris entre 0 et 255.

Il permet principalement d'identifier l'origine de l'erreur.

10. Exemple d'utilisation

```
CREATE TRIGGER TR_Verification
ON Employe
AFTER UPDATE
AS
BEGIN
    IF UPDATE(Salaire)
    BEGIN
        RAISERROR('Modification interdite',16,1);
        ROLLBACK;
    END
END;
```

Ici :

- un utilisateur modifie un salaire ;
- le trigger se déclenche automatiquement ;
- un message d'erreur est affiché ;
- la modification est annulée.

★ Les pièges

Un trigger ne s'exécute jamais avec EXEC

Il est lancé automatiquement.

INSERTED et DELETED ne sont pas de vraies tables

Elles n'existent que pendant l'exécution du trigger.

UPDATE()

Ne modifie rien.

Elle vérifie uniquement si une colonne a été modifiée.

ROLLBACK

Annule également l'instruction qui a déclenché le trigger.

★ À connaître par cœur

Commande	Utilité
CREATE TRIGGER	Créer un trigger
AFTER INSERT	Après une insertion
AFTER UPDATE	Après une modification
AFTER DELETE	Après une suppression
INSERTED	Nouvelles valeurs
DELETED	Anciennes valeurs
UPDATE(Colonne)	Vérifie si une colonne a été modifiée
ROLLBACK	Annule la transaction
RAISERROR	Affiche une erreur personnalisée
@@ERROR	Retourne le code de la dernière erreur

🎯 Le conseil "prof"

Si tu devais retenir 4 notions essentielles de ce chapitre, ce seraient :

1. Un **trigger** s'exécute automatiquement.
2. INSERTED contient les nouvelles données, DELETED les anciennes.
3. UPDATE(Colonne) permet de savoir si une colonne a été modifiée.
4. ROLLBACK permet de refuser complètement une opération.